

Dominators

12 May, 2026

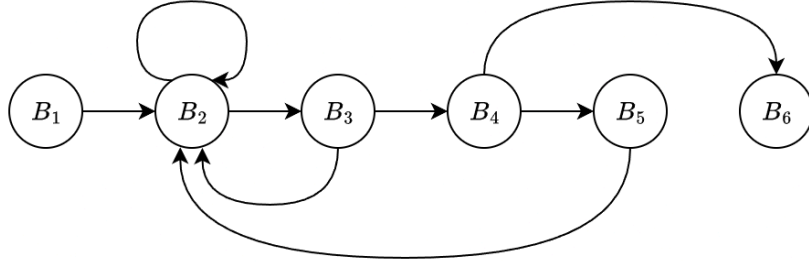


Figure 1: Flow graph of example 2 of basic blocks

Data flow analysis

- # It is a process of collecting information about the variables that are used in the program.
- # The information collected at various points can be related using simple set equations.
- # A typical equation has the form:

$$\text{out}[s] = \text{gen}[s] \cup (\text{in}[s] - \text{kill}[s])$$

and can be read as *the information at the end of a statement is either generated within the statement or enters at the beginning of the statement and is not killed as control flows through the statement.*

Point

- # By a *point* in a program, we mean the position before or after any statement. We say control reaches the *point before* a statement when the statement is about to be executed and the *point after* when the statement has been executed.

Path

- # A path from p_1 to p_n is a sequence of points $p_1, p_2, \dots, p_n$ such that for each i between 1 and $n - 1$ either:
 - i. p_i is the point immediately preceding a statement and p_{i+1} is the point immediately following the statement in the same block. In this case, the *point after* a statement s is the same as the *point before* the statement $s + 1$.
 - ii. p_i is the end of some block and p_{i+1} is the beginning of the successor block. In this case, the *point after* the final statement of a block is followed by the *point before* the first statement of the successor block.

Definition

- # A *definition* of a variable x is a statement that assigns or may assign a value to x .

Reach

- # A definition d is said to *reach* a point p if there is a path from the point immediately following d to p such that d is not *killed* along the path.

Kill

We *kill* a definition of a variable a if between two points, along the path there is another definition of a .

Gen[s]

We define $\text{gen}[s]$ as the set of *definitions* generated by statement s .

We say *definition* d is in $\text{gen}[s]$ if d reaches the end of s independent of whether it reaches the beginning of s .

Another definition of $\text{gen}[s]$ could be: *definition* d is in $\text{gen}[s]$ if d is not *killed* by another subsequent *definition* in the same block, and is thus “visible” at the end of the block.

Kill[s]

We say $\text{kill}[s]$ is the set of definitions that never reaches the end of s even if they reach the beginning.

#

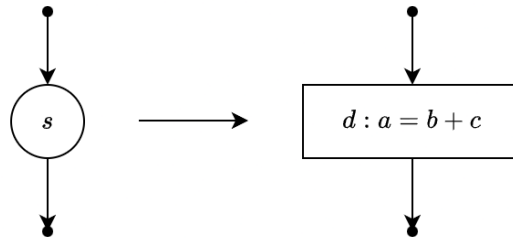


Figure 2: Definition of a statement

The assignment is a definition of a , say definition d . Then d is the only definition sure to reach the end of the statement regardless of whether it reaches the beginning.

$$\text{gen}[s] = \{d\}$$

d kills all other definitions, so we write

$$\text{kill}[s] = D_a - \{d\}$$

where D_a is the set of all definitions in the program for variable a .

#

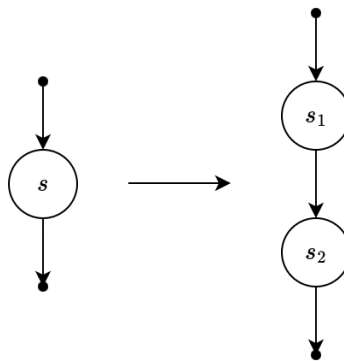


Figure 3: Definition of consecutive statements

We next consider a consecutive set of statements s_1 and s_2 . Consider the definition d , if it is generated by s_2 then it is surely generated by s . If d is generated by s_1 , it will reach the end of s provided it is not killed by s_2 . Then we write:

$$\begin{aligned} \text{gen}[s] &= \text{gen}[s_2] \cup (\text{gen}[s_1] - \text{kill}[s_2]) \\ \text{kill}[s] &= \text{kill}[s_2] \cup (\text{kill}[s_1] - \text{gen}[s_2]) \end{aligned}$$

#

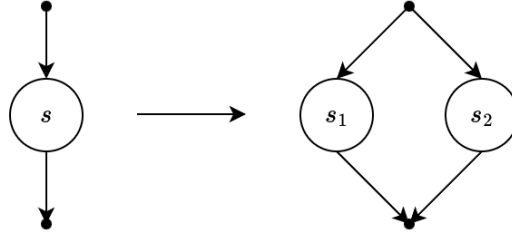


Figure 4: Definition of *if* statement

For the *if* statment, if either branch of the *if* generates a statement, then that definition reaches the end of the statemnt. Thus, to kill definition d , it must be killed along either branch so that

$$\begin{aligned} \text{gen}[s] &= \text{gen}[s_1] \cup \text{gen}[s_2] \\ \text{kill}[s] &= \text{kill}[s_1] \cap \text{kill}[s_2] \end{aligned}$$

If defintion d is generated within s_1 , then it reaches both the end of s_1 and s . Conversely, if d is generated within s , it can be generated within s_1 . We conclude

$$\text{gen}[s] = \text{gen}[s_1]$$

by a similar set of arguments, we can say

$$\text{kill}[s] = \text{kill}[s_1]$$

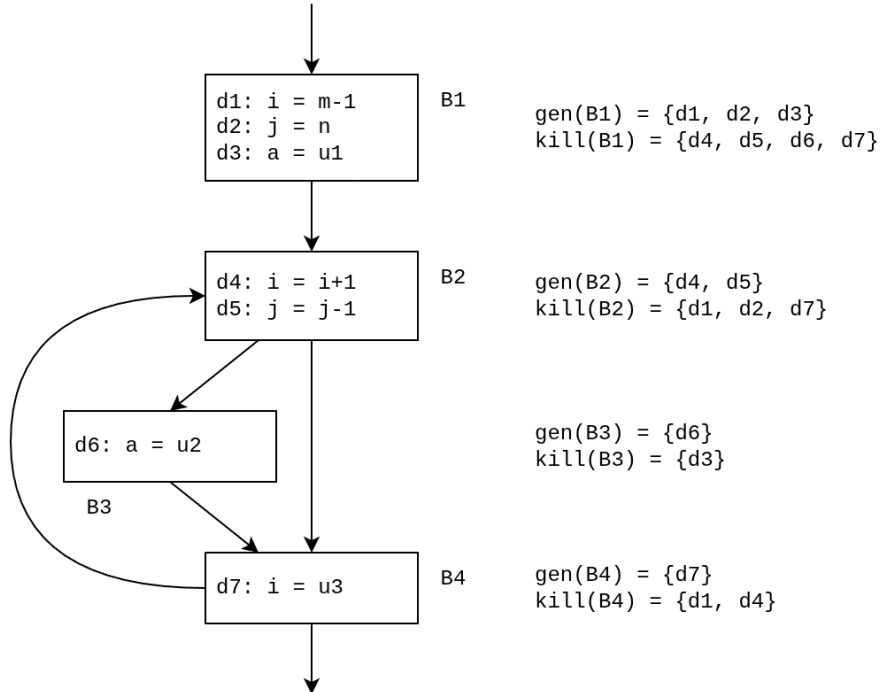


Figure 5: Flow graph for reaching definitions

Dominators

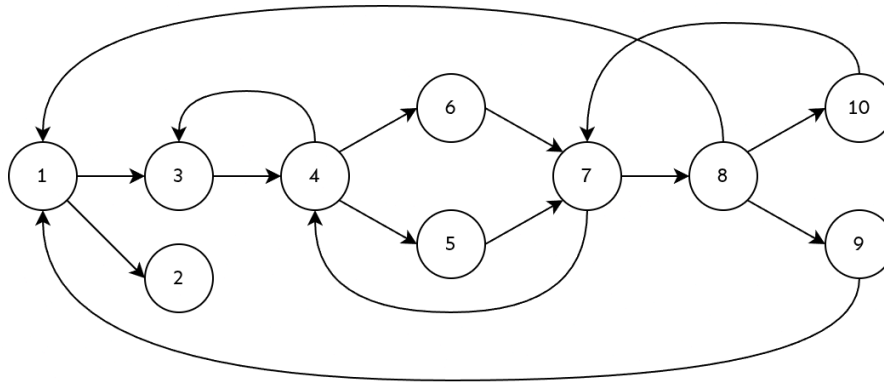


Figure 6: Flow graph

- # We say node d of a flow graph dominates node n (written as $d \text{ DOM } n$), if every path from the initial node to the flow graph to n goes through d .
- # In the above example:
 - The initial node dominates every node.
 - Node 2 dominates only itself.
 - Node 3 dominates all but 1 and 2.
 - Node 4 dominates all but 1, 2 and 3.
 - Node 5 and 6 dominate only themselves.
 - Node 7 dominates 7, 8, 9 and 10.
 - Node 8 dominates 8, 9 and 10.
 - Node 9 and 10 dominate only themselves.
- # Dominance is reflexive, i.e. $a \text{ DOM } a$ holds.
- # Dominance is also transitive, i.e. $a \text{ DOM } b$ and $b \text{ DOM } c \Rightarrow a \text{ DOM } c$.
- # Dominance is anti-symmetric, i.e. $a \text{ DOM } b$ and $b \text{ DOM } a \Rightarrow a = b$
- # Dominance is a reflexive partial order. The dominance of each node are heirarchically ordered by DOM relation.
- # E.g. The dominators of 9 are 1, 3, 4, 7, 8 and 9. It can be found that $1 \text{ DOM } 3 \text{ DOM } 4 \text{ DOM } 7 \text{ DOM } 8 \text{ DOM } 9$. 8 is called the immediate dominator of 9.

Back edges

- # We can search for edges in the flow graph whose heads dominate their tails. If $A \rightarrow B$ is an edge, B is the head and A is the tail. We call such edges - back edges.
- # Example: There is an edge $7 \rightarrow 4$ and $4 \text{ DOM } 7$. Similarly, $10 \rightarrow 7$ is an edge and $7 \text{ DOM } 10$. Note that these are exactly the edges that can be seen as forming loop in the flow graph.

Natural loops

- # Given a back edge $n \rightarrow d$, we define the natural loop of the edge to be d plus the set of nodes that can reach n without going through d . Node d is called the *header* of the loop.
- # Example: The natural loop containing 7, 8 and 10 since 8 and 10 are all those nodes that can reach 10 without going through 7. The natural loop of $9 \rightarrow 1$ is the entire flow graph.

Depth-First Spanning Tree (DFST)

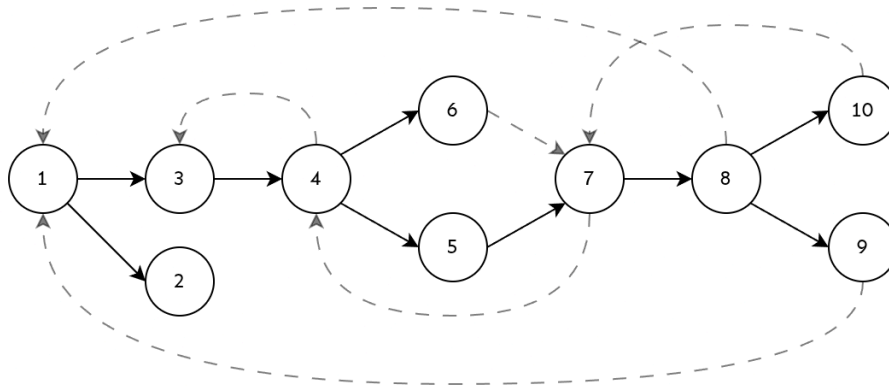


Figure 7: Depth-first spanning tree

- # A depth-first search of a graph visits all nodes in the graph once. The route of a depth-first search forms a *depth-first spanning tree*.
- # There are three types of edges in a DFST:
 1. **Advancing edges:** Edges that go from a node to any of its descendent are called *advancing edges*. All the edges represented by solid arrows are *advancing edges*.
 2. **Retreating edges:** Edges that go from a node to itself or any of its ancestors are called *retreating edges*. All the edges represented by dashed arrows except $6 \rightarrow 7$ are *retreating edges*.
 3. **Cross edges:** Edges that connect two nodes which do not follow an ancestor-descendent relationship are called *cross edges*. $6 \rightarrow 7$ is a *cross edge*.

Reducible flow graph

- # A flow graph is *reducible* if the *retreating edges* in any DFST are all *back edges*. For example, the retreating edges in Figure 7 are:

$$4 \rightarrow 3 \mid 7 \rightarrow 4 \mid 8 \rightarrow 1 \mid 9 \rightarrow 1 \mid 10 \rightarrow 7$$

which are all *back edges*.

- # All back edges are *retreating edges*, but if a flow graph is *nonreducible*, its DFST has additional *retreating edges* that are not *back edges*, i.e. even if we remove all the *back edges* of a *nonreducible* flow graph, the graph will still be cyclic.
- # Alternatively, a flow graph is *reducible* if and only if we can partition the edges into two disjoint groups often called the forward edges and back edges with the following two properties:
 1. The forward edges form an acyclic graph in which every node can be reached from the initial node of G (basically, advancing edges $\cup$ cross edges forms a directed acyclic graph).
 2. The back edges consist only of edges whose heads dominate their tails (basically, retreating edges = back edges).
- # Consider the following flow graph. $3 \rightarrow 2$ is *retreating edge*, but it is not a *back edge*. Alternatively, it has no back edges, but it still contains a cycle. So, this is a *nonreducible* flow graph.

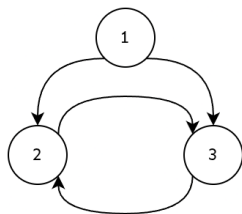


Figure 8: A *nonreducible* flow graph

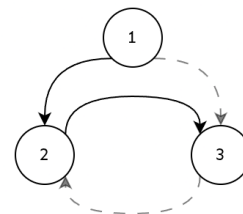


Figure 9: Corresponding DFST